



香港中文大學(深圳)
The Chinese University of Hong Kong, Shenzhen

Introduction to Computer Engineering: Programming Applications

Lecture 7 Object Oriented Programming II

Prof. Junhua Zhao
School of Science and Engineering

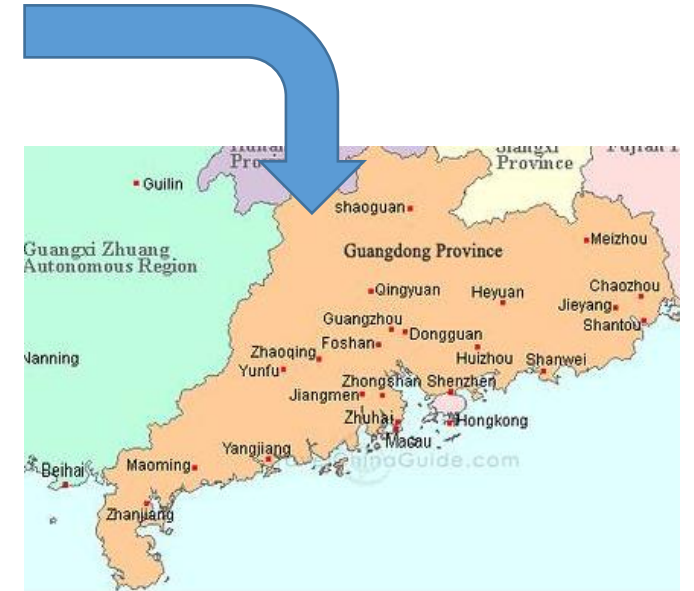
Inheritance

- The object-oriented programming couples data and methods together into objects
- The object oriented approach combines the power of the functional programming with an added dimension that integrates data with operations into objects
- Object-oriented programming (OOP) allows you to define new classes from existing classes. This is called **inheritance**
- Inheritance extends the power of the object-oriented paradigm by adding an important and powerful feature for reusing software

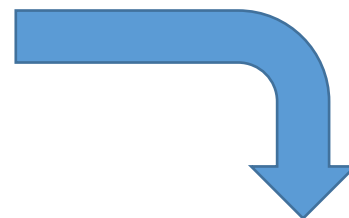
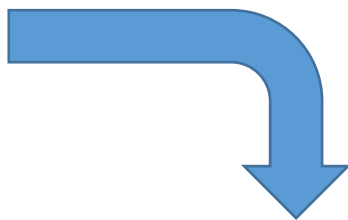
Superclass and subclass

- **Inheritance** enables you to define a general class (a **superclass**) and later extend it to more specialized classes (**subclasses**)
- You use a class to model objects of the same type. Different classes may have some **common properties and behaviours** that you can generalize in a class
- **Inheritance** enables you to define a general class and later extend it to define more specialized classes
- The specialized classes **inherit** the properties and methods from the general class.

Example: Human -> Chinese -> Cantonese

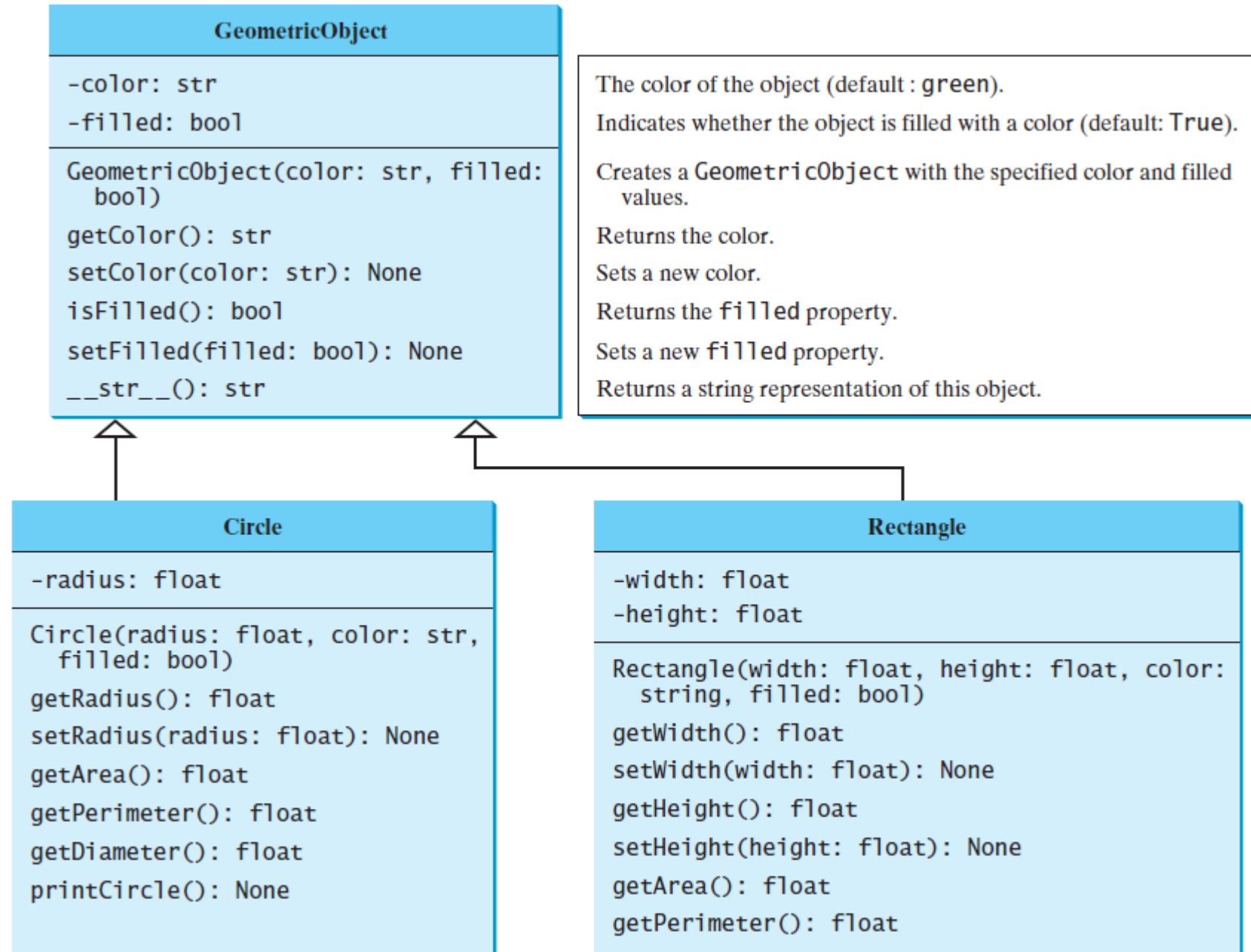


Example: Human -> Star -> Movie Star



今天的心情是
大不同，
啊大不同

Geometric Object and two of its subclasses



The code for GeometricObject

<pre>class GeometricObject: def __init__(self, color = "green", filled = True): self.__color = color self.__filled = filled def getColor(self): return self.__color def setColor(self, color): self.__color = color def isFilled(self): return self.__filled def setFilled(self, filled): self.__filled = filled def __str__(self): return "color: " + self.__color + \ " and filled: " + str(self.__filled)</pre>	GeometricObject class initializer data fields getColor setColor isFilled
---	--

The code for Circle class

- A subclass inherits accessible data fields and methods from its superclass, but it can also have other data fields and methods

```
from GeometricObject import GeometricObject
import math # math.pi is used in the class
```

```
class Circle(GeometricObject):
    def __init__(self, radius):
        super().__init__()
        self.__radius = radius

    def getRadius(self):
        return self.__radius

    def setRadius(self, radius):
        self.__radius = radius

    def getArea(self):
        return self.__radius * self.__radius * math.pi

    def getDiameter(self):
        return 2 * self.__radius

    def getPerimeter(self):
        return 2 * self.__radius * math.pi

    def printCircle(self):
        print(self.__str__() + " radius: " + str(self.__radius))
```


Inheritance syntax

- The **Circle** class is derived from the **GeometricObject** class, based on the following syntax

```
subclass      superclass
   \          /
    class Circle(GeometricObject):
```

- **Circle** class inherits the **GeometricObject** class, thus inheriting the methods **getColor**, **setColor**, **isFilled**, **setFilled**, and **__str__**
- The **printCircle** method invokes the **__str__()** method defined to obtain properties defined in the superclass

The code for rectangle class

```
from GeometricObject import GeometricObject
```

```
class Rectangle(GeometricObject):  
    def __init__(self, width = 1, height = 1):  
        super().__init__()   
        self.__width = width  
        self.__height = height  
  
    def getWidth(self):  
        return self.__width  
  
    def setWidth(self, width):  
        self.__width = width  
  
    def getHeight(self):  
        return self.__height  
  
    def setHeight(self, height):  
        self.__height = self.__height  
  
    def getArea(self):  
        return self.__width * self.__height  
  
    def getPerimeter(self):  
        return 2 * (self.__width + self.__height)
```

extend superclass
initializer
superclass initializer

methods

The code for testing Circle and Rectangle

```
from CircleFromGeometricObject import Circle
from RectangleFromGeometricObject import Rectangle

def main():
    circle = Circle(1.5)
    print("A circle", circle)
    print("The radius is", circle.getRadius())
    print("The area is", circle.getArea())
    print("The diameter is", circle.getDiameter())

    rectangle = Rectangle(2, 4)
    print("\nA rectangle", rectangle)
    print("The area is", rectangle.getArea())
    print("The perimeter is", rectangle.getPerimeter())

main() # Call the main function
```

```
A circle color: green and filled: True
The radius is 1.5
The area is 7.06858347058
The diameter is 3.0
```

```
A rectangle color: green and filled: True
The area is 8
The perimeter is 12
```

Some more information about super and sub-class

- A subclass is **not a subset** of its superclass; In fact, a subclass usually contains **more information and methods** than its superclass
- Inheritance models the is-a relationships, but **not all** is-a relationships should be modelled using inheritance
- Do not **blindly extend a class** just for the sake of reusing methods. For example, it makes no sense for a Tree class to extend a Person class, even though they share common properties such as height and weight. A subclass and its superclass **must have the is-a relationship**

Practice

```
class A:
    def __init__(self, i = 0):
        self.i = i

class B(A):
    def __init__(self, j = 0):
        self.j = j

def main():
    b = B()
    print(b.i)
    print(b.j)

main() # Call the main function
```

What is the problem with the above code?

Overriding methods

- A subclass **inherits** methods from a superclass
- Sometimes it is necessary for the subclass to modify the implementation of a method defined in the superclass. This is referred to as **method overriding**

Example

- The `__str__` method in the `GeometricObject` class returns the string describing a geometric object. This method can be overridden to return the string describing a circle
- The `__str__()` method is defined in the `GeometricObject` class and modified in the `Circle` class. Both methods can be used in the `Circle` class. To invoke the `__str__` method defined in the `GeometricObject` class from the `Circle` class, use `super().__str__()`

```
class Circle(GeometricObject):
    # Other methods are omitted

    # Override the __str__ method defined in GeometricObject
    def __str__(self):
        return super().__str__() + " radius: " + str(radius)           __str__ in superclass
```


Practice

What would be the output of the following program?

```
class A:
    def __init__(self, i = 0):
        self.i = i

    def m1(self):
        self.i += 1

class B(A):
    def __init__(self, j = 0):
        super().__init__(3)
        self.j = j

    def m1(self):
        self.i += 1

def main():
    b = B()
    b.m1()
    print(b.i)
    print(b.j)

main() # Call the main function
```

The object class

- Every class in Python is descended from the object class
- The object class is defined in the Python library. If no inheritance is specified when a class is defined, its superclass is object by default

```
class ClassName:  
    ...
```

Equivalent

```
class ClassName(object):  
    ...
```

Methods of the object class

- The `__new__()` method is automatically invoked when an object is constructed. This method then invokes the `__init__()` method to initialize the object. Normally you should only override the `__init__()` method to initialize the data fields defined in the new class
- The `__str__()` method returns a string description for the object
- Usually you should override the `__str__()` method so that it returns an informative description for the object

What is the output of this program?

```
class A:
    def __init__(self, i = 0):
        self.i = i

    def m1(self):
        self.i += 1

    def __str__(self):
        return 'The content of this object is:' + str(self.i)

x = A(8)
print(x)
```

What is the output of this program?

```
class A:
    def __new__(self):
        print("A's __new__() invoked")

    def __init__(self):
        print("A's __init__() invoked")

class B(A):
    def __new__(self):
        print("B's __new__() invoked")

    def __init__(self):
        print("B's __init__() invoked")

def main():
    b = B()
    a = A()

main() # Call the main function
```

What is the output of this program?

```
class A:
    def __new__(self):
        self.__init__(self)
        print("A's __new__() invoked")

    def __init__(self):
        print("A's __init__() invoked")

class B(A):
    def __new__(self):
        self.__init__(self)
        print("B's __new__() invoked")

    def __init__(self):
        print("B's __init__() invoked")

def main():
    b = B()
    a = A()

main() # Call the main function
```

Polymorphism and dynamic binding

- **Polymorphism** means that the objects of different classes can be passed as arguments to the same function
- A method may be implemented in several classes along the inheritance chain
- Python decides which method is invoked at runtime. This is known as **dynamic binding**

Polymorphism

- The **inheritance** relationship enables a subclass to inherit members from its superclass with additional new members
- A subclass is a **specialization** of its superclass; every instance of a subclass is also an instance of its superclass, but not vice versa
- Therefore, when two objects share some common members, they can both be passed as arguments to the same function

Example

```
from CircleFromGeometricObject import Circle
from RectangleFromGeometricObject import Rectangle

def main():
    # Display circle and rectangle properties

    c = Circle(4)
    r = Rectangle(1, 3)
    displayObject(c)
    displayObject(r)
    print("Are the circle and rectangle the same size?",
          isSameArea(c, r))

# Display geometric object properties
def displayObject(g):
    print(g.__str__())

# Compare the areas of two geometric objects
def isSameArea(g1, g2):
    return g1.getArea() == g2.getArea()

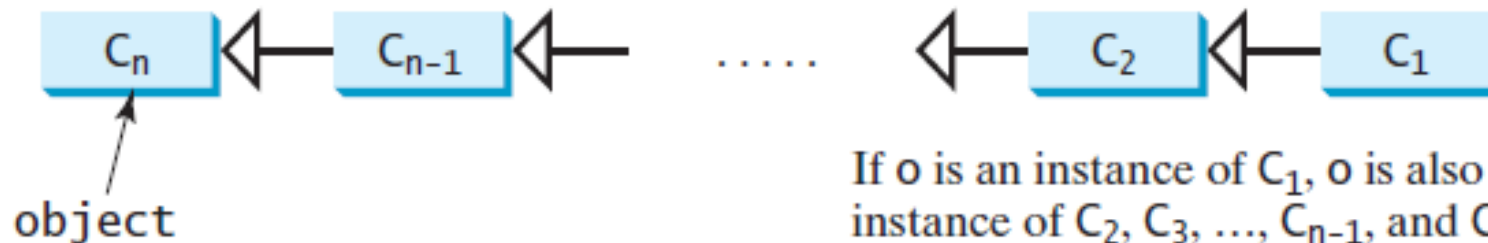
main() # Call the main function
```

Output

```
color: green and filled: True radius: 4  
color: green and filled: True width: 1 height: 3  
Are the circle and rectangle the same size? False
```

Dynamic binding

- Dynamic binding works as follows: Suppose an object o is an instance of classes $C_1, C_2, \dots, C_{n-1}$, and C_n , where C_1 is a subclass of C_2 , C_2 is a subclass of C_3 , ..., and C_{n-1} is a subclass of C_n
- That is, C_n is the most general class, and C_1 is the most specific class
- In Python, C_n is the object class
- If o invokes a method p , Python searches the implementation for the method p in $C_1, C_2, \dots, C_{n-1}$, and C_n , in this order, until it is found



Example

- What would be the output of this program?

```
class C1:
    def __init__(self):
        self.f = 1

    def output(self):
        print('In C1, the f is:', self.f)

class C2(C1):
    def __init__(self):
        self.f = 2

    def output(self):
        print('In C2, the f is:', self.f)

class C3(C2):
    def __init__(self):
        self.f = 3

class C4(C3):
    def __init__(self):
        self.f = 4

a=C4()
print(a.f)
a.output()
```

Example

```
class Student:
    def __str__(self):
        return "Student"

    def printStudent(self):
        print(self.__str__())

class GraduateStudent(Student):
    def __str__(self):
        return "Graduate Student"

a = Student()
b = GraduateStudent()
a.printStudent()
b.printStudent()
```

Question

- Suppose you want to modify the displayObject function in previous example to perform the following tasks:
 - Display the area and perimeter of a Circle or Rectangle instance
 - Display the diameter if the instance is a Circle, and the width and height if the instance is a Rectangle

Does this program work?

```
def displayObject(g):  
    print("Area is", g.getArea())  
    print("Perimeter is", g.getPerimeter())  
    print("Diameter is", g.getDiameter())  
    print("Width is", g.getWidth())  
    print("Height is", g.getHeight())
```

Isinstance() function

- The `isinstance()` function can be used to determine whether an object is an instance of a class
- This function determines whether an object is an instance of a class by using the following syntax

```
isinstance(object, ClassName)
```



```
from CircleFromGeometricObject import Circle
from RectangleFromGeometricObject import Rectangle

def main():
    # Display circle and rectangle properties
    c = Circle(4)
    r = Rectangle(1, 3)
    print("Circle...")
    displayObject(c)
    print("Rectangle...")
    displayObject(r)

# Display geometric object properties
def displayObject(g):
    print("Area is", g.getArea())
    print("Perimeter is", g.getPerimeter())

    if isinstance(g, Circle):
        print("Diameter is", g.getDiameter())
    elif isinstance(g, Rectangle):
        print("Width is", g.getWidth())
        print("Height is", g.getHeight())

main() # Call the main function
```

```
Circle...
Area is 50.26548245743669
Perimeter is 25.132741228718345
Diameter is 8
Rectangle...
Area is 3
Perimeter is 8
Width is 1
Height is 3
```

Practice

```
class Person:
    def getInfo(self):
        return "Person"

    def printPerson(self):
        print(self.getInfo())

class Student(Person):
    def getInfo(self):
        return "Student"

Person().printPerson()
Student().printPerson()
```

(a)

```
class Person:
    def __getInfo(self):
        return "Person"

    def printPerson(self):
        print(self.__getInfo())

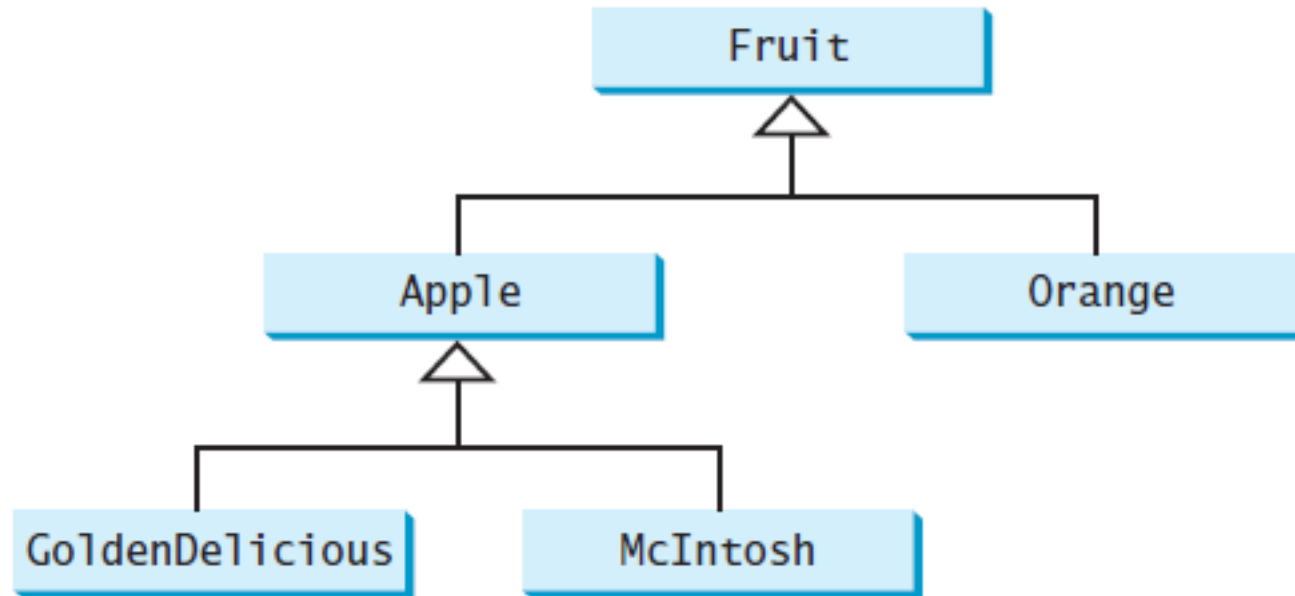
class Student(Person):
    def __getInfo(self):
        return "Student"

Person().printPerson()
Student().printPerson()
```

(b)

What would be the outputs?

Practice



Assume that the following statements are given:

```
goldenDelicious = GoldenDelicious()  
orange = Orange()
```



Questions

- (a) Is `goldenDelicious` an instance of `Fruit`?
- (b) Is `goldenDelicious` an instance of `Orange`?
- (c) Is `goldenDelicious` an instance of `Apple`?
- (d) Is `goldenDelicious` an instance of `GoldenDelicious`?
- (e) Is `goldenDelicious` an instance of `McIntosh`?
- (f) Is `orange` an instance of `Orange`?
- (g) Is `orange` an instance of `Fruit`?
- (h) Is `orange` an instance of `Apple`?
- (i) Suppose the method `makeAppleCider` is defined in the `Apple` class. Can `goldenDelicious` invoke this method? Can `orange` invoke this method?
- (j) Suppose the method `makeOrangeJuice` is defined in the `Orange` class. Can `orange` invoke this method? Can `goldenDelicious` invoke this method?

Practice

(**The Compound class**) Calculating Molar Mass of a Compound: In chemistry, the molar mass of a compound is the sum of the atomic masses of its constituent elements, multiplied by their respective counts in the compound. This program will calculate the molar mass of simple compounds based on their molecular formula.

The class contains:

- Two data fields named **formula** and **element_counts**.
- A method named **parse_formula()** that parses the chemical formula and returns a dictionary with element symbols as keys and their counts as values.
- A method named **calculate_molar_mass()** that calculates the molar mass of the compound based on atomic masses and element counts.

The compound class

```
# Atomic masses of elements (in g/mol)
atomic_masses = { dictionary
    "H": 1.008,
    "C": 12.011,
    "O": 15.999,
    "N": 14.007,
    "Cl": 35.453,
    "Na": 22.990,
    # Add more elements as needed
}
```

```
class Compound:
    def __init__(self, formula):
        self.formula = formula
        self.element_counts = self.parse_formula()

    def parse_formula(self):
        """
        Parses the chemical formula and returns a dictionary with element symbols as keys
        and their counts as values.
        Example: H2O -> {'H': 2, 'O': 1}
        """
        import re analysing str package
        element_counts = {}
        # Regex pattern to capture elements and their counts
        pattern = r"([A-Z][a-z]*) (\d*)"
        matches = re.findall(pattern, self.formula)

        for element, count in matches:
            count = int(count) if count else 1 # Default count is 1 if not specified
            if element in element_counts:
                element_counts[element] += count
            else:
                element_counts[element] = count

        return element_counts

    def calculate_molar_mass(self):
        """
        Calculates the molar mass of the compound based on atomic masses and element counts.
        """
        molar_mass = 0.0
        for element, count in self.element_counts.items():
            if element in atomic_masses:
                molar_mass += atomic_masses[element] * count
            else:
                raise ValueError(f"Atomic mass for element '{element}' not found.")
                error/mistake detected by python
        return molar_mass (print variable that are not found / turn non-num str to numbers)

    overriding
    def __str__(self):
        return self.formula + ' - Molar Mass: ' + str(self.calculate_molar_mass()) + ' g/mol'
```

Use of the compound class

```
try:
    water = Compound("H2O")
    print(water)

    carbon_dioxide = Compound("CO2")
    print(carbon_dioxide)

    sodium_chloride = Compound("NaCl")
    print(sodium_chloride)

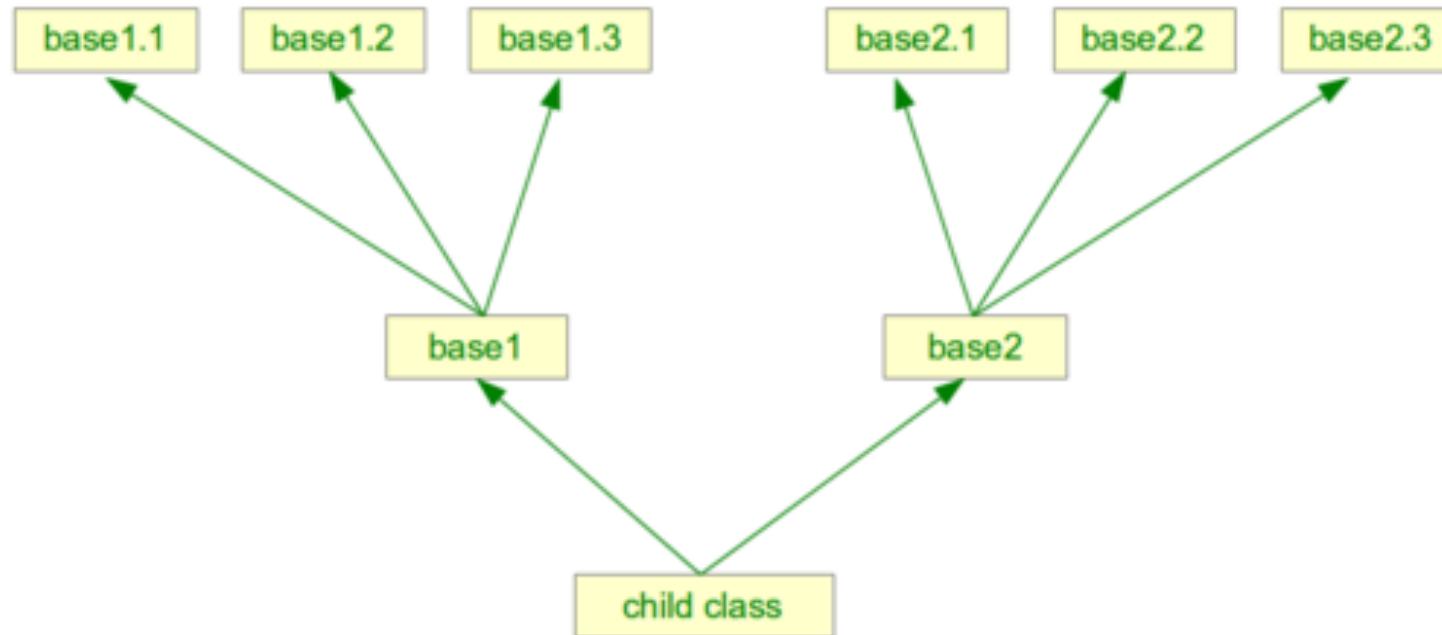
except ValueError as e:
    print(e)
```

Multiple Inheritance

1 Subclass from >1 superclass

- In Python, we can define new class from multiple classes
- This is called **multiple inheritance**
- **Multiple inheritance** is a feature in which a class can **inherit data fields and methods from more than one parent class**

Inheritance Tree



- The inheritance relationship in Python can be represented by a tree structure

Example

```
class A():  
    def __init__(self, a=100):  
        self.a=a
```

```
class B():  
    def __init__(self, b=200):  
        self.b=b
```

```
class C(A, B):  
    def __init__(self, a, b, c=300):  
        super().__init__(a) 1 a=1 b=2 c=3  
        super().__init__(b) 2 a=1 b=2 c=3  
        self.c=c 3
```

Super() always refers to the 1st superclass

Inheriting everything from both A and B

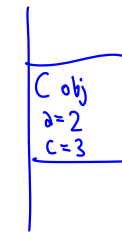
1st super init
→ 1st super parent name is C

```
    def output(self):  
        print(self.a)  
        print(self.c)  
        print(self.b)
```

```
def main():  
    c = C(1, 2, 3)  
    c.output()
```

```
main()
```

Output:
2
3
Error



Example

```
class A():
    def __init__(self, a=100):
        self.a=a

class B():
    def __init__(self, b=200):
        self.b=b

class C(A, B):
    def __init__(self, a, b, c=300):
        A.__init__(self, a)
        B.__init__(self, b)
        self.c=c

    def output(self):
        print(self.a)
        print(self.c)
        print(self.b)

def main():
    c = C(1, 2, 3)
    c.output()

main()
```

```
class star:
    def __init__(self, name, gender):
        print('Star init...')
        self.name = name
        self.gender = gender
```

ยี่สิบสี่ - ยี่สิบห้า (สิบสาม)

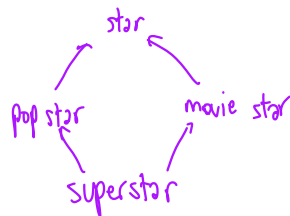
Should use single inheritance

```
class popStar(star): inherit star
    def __init__(self, albumList):
        print('popStar init...')
        star.__init__(self, 'Stephen Chow', 'Male')
        self.albumList = albumList
```

```
def outputAlbum(self):
    print(self.name, 'has published the following albums:')
    for key in self.albumList:
        print('The sold copies of album <'+key+'> is', self.albumList[key])
```

```
class movieStar(star): inherit star
    def __init__(self, movieList):
        print('movieStar init...')
        star.__init__(self, 'Stephen Chow', 'Male')
        self.movieList = movieList
```

```
def outputMovie(self):
    print(self.name, 'has participated in the following movies:')
    for key in self.movieList:
        print('The income of movie <'+key+'> is', self.movieList[key])
```



Calling method from parents, just use init from parents

```
class superStar(movieStar, popStar):
    def __init__(self, albumList, movieList):
        print('superStar init...')
        popStar.__init__(self, albumList)
        movieStar.__init__(self, movieList)
```

```
def careerPerformance(self):
    print(self.name+ ' has a very successful career.')
    popStar.outputAlbum(self)
    movieStar.outputMovie(self)
```

```
def main():
    albumList = {'I am a singer':100, 'Hahaha':200, 'Gee':300}
    movieList = {'Kungfu':2000000, 'Shaolin Soccer':20000000, 'Mermaid':230909230}
    s = superStar(albumList, movieList)
    s.careerPerformance()
```

main()